

Теория параллелизма

Отчет

Задание № 7

Сергеев Егор Валерьевич, 23932

г. Новосибирск

2025 г.

Цель работы: попробовать на практике использование openACC на примере решения уравнения теплопроводности разностной схемой, протестировать ускорение на нескольких ядрах cpu и на gpu.

Компилятор: pgc++

Профилировщик: Nsight Systems

Замеры времени: `std::chrono::high_resolution_clock` (итоговое время в секундах)

Выполнение на CPU

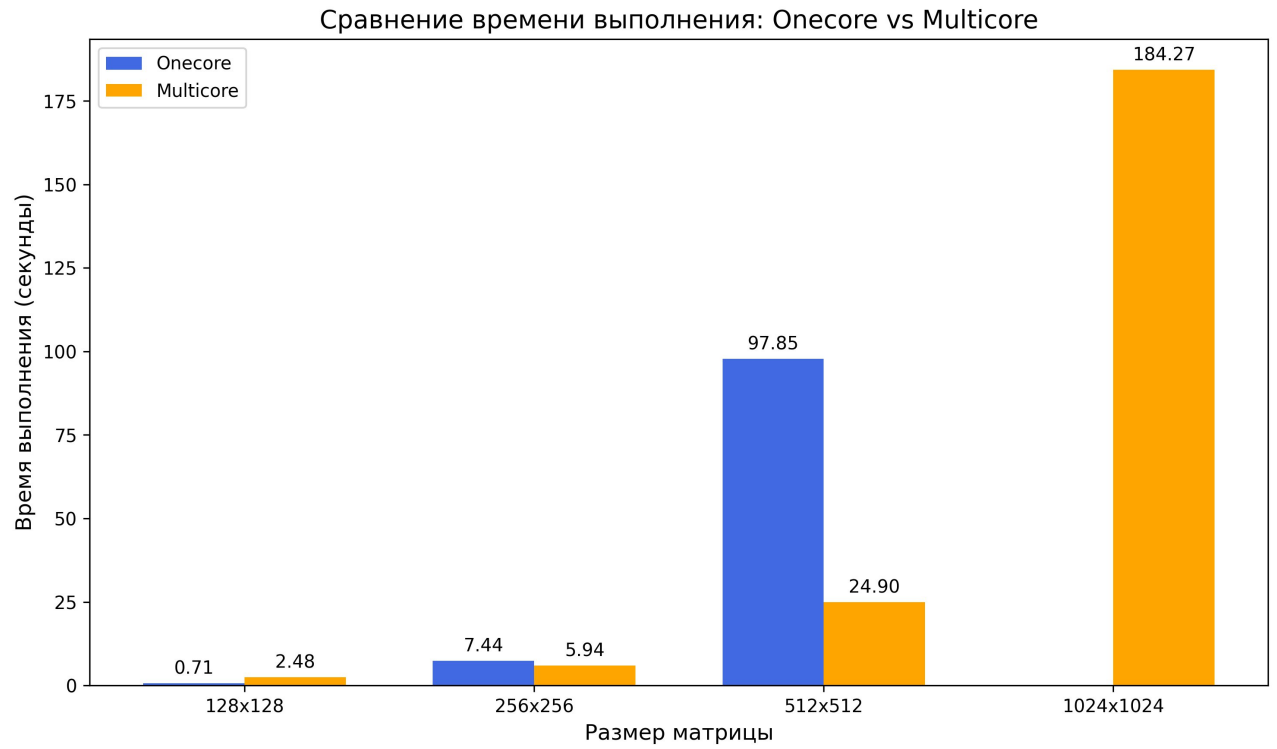
CPU-onescore

Размер сетки	Время выполнения	Точность	Количество итераций
128*128	0.708994 сек	4.79656e-08	40000
256*256	7.43616 сек	5.82721e-07	110000
512*512	97.852 сек	9.92435e-07	340000
1024*1024	> 5 мин	-	1000000

CPU-multicore

Размер сетки	Время выполнения	Точность	Количество итераций
128*128	2.4795 сек	4.79656e-08	40000
256*256	5.94472 сек	5.82721e-07	110000
512*512	24.9016 сек	9.92435e-07	340000
1024*1024	184.265 сек	1.36929e-06	1000000

Диаграмма сравнения время работы CPU-one и CPU-multi

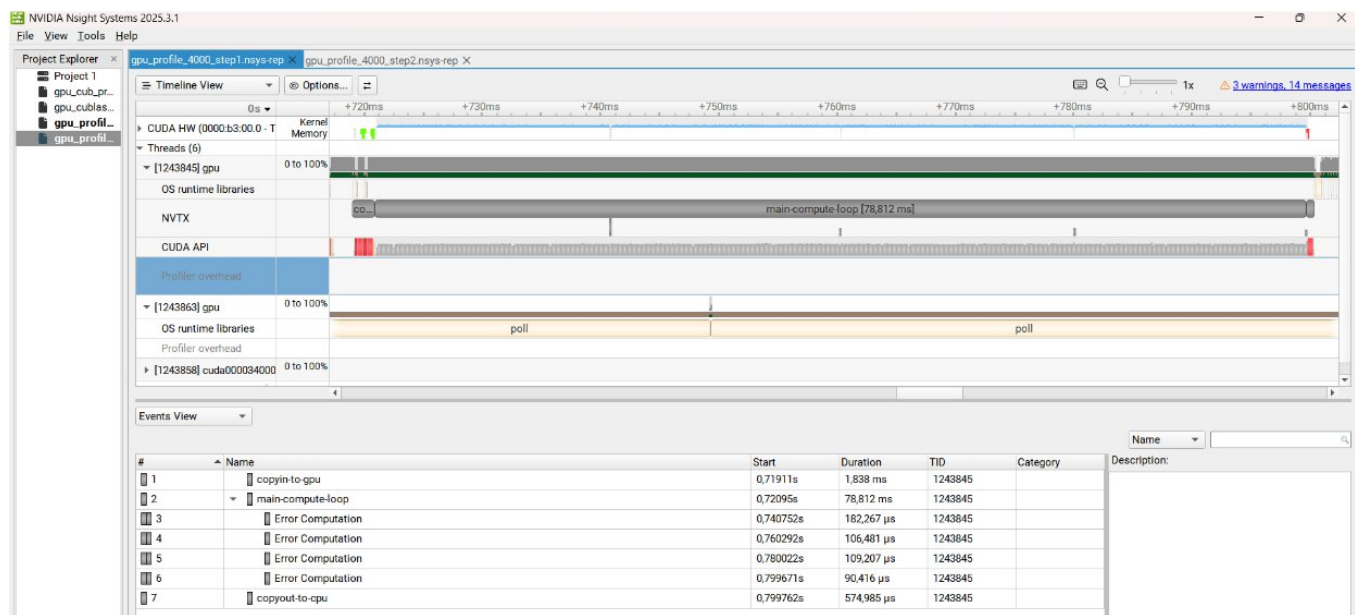


Выполнение на GPU

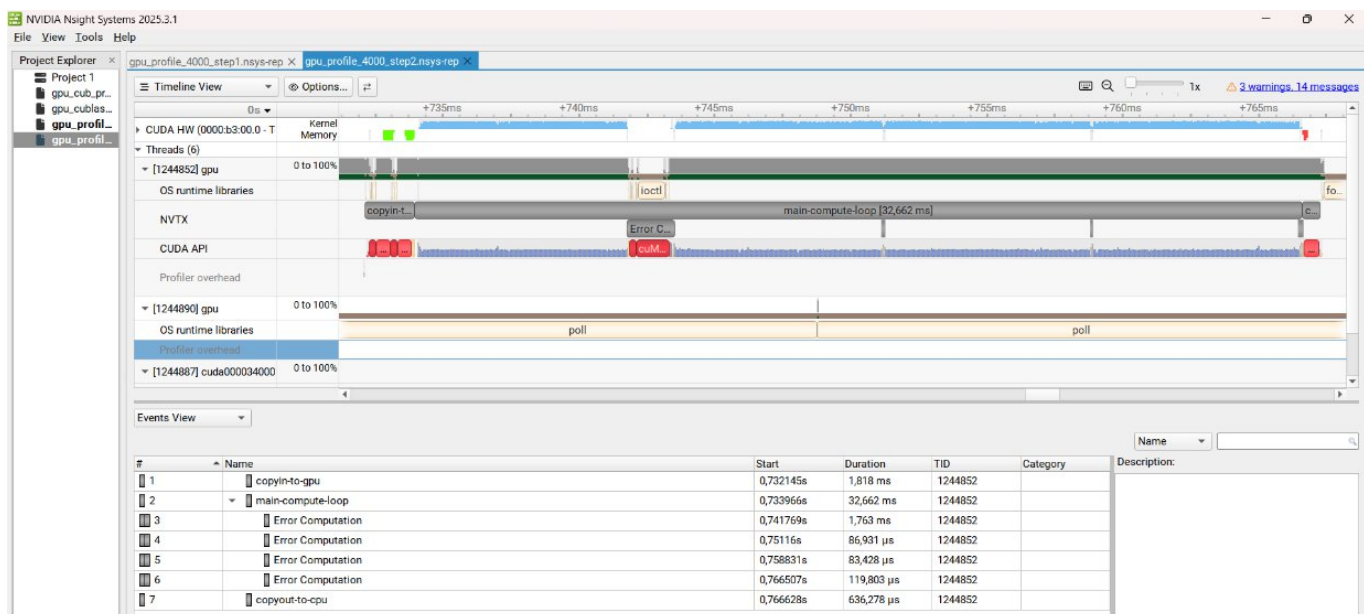
Этапы оптимизации на сетке 512*512

Этап №	Время выполнения	Ошибка	Максимальное количество итераций	Комментарии
1	48.8017 сек.	1.36929e-06	1000000	Базовое применение openACC для ускорения расчётов
2	26.1976 сек.	1.36929e-06	1000000	Использование async при использовании openACC

Профилрование этап 1



Профилрование этап 2



GPU - оптимизированный вариант

Размер сетки	Время выполнения	Точность	Количество итераций
128*128	0.125 сек	4.7965e-08	40000
256*256	0.32 сек	5.82721e-07	110000
512*512	1.806 сек	9.92435e-07	340000
1024*1024	25.411 сек	1.36929e-06	1000000

Размер: 10

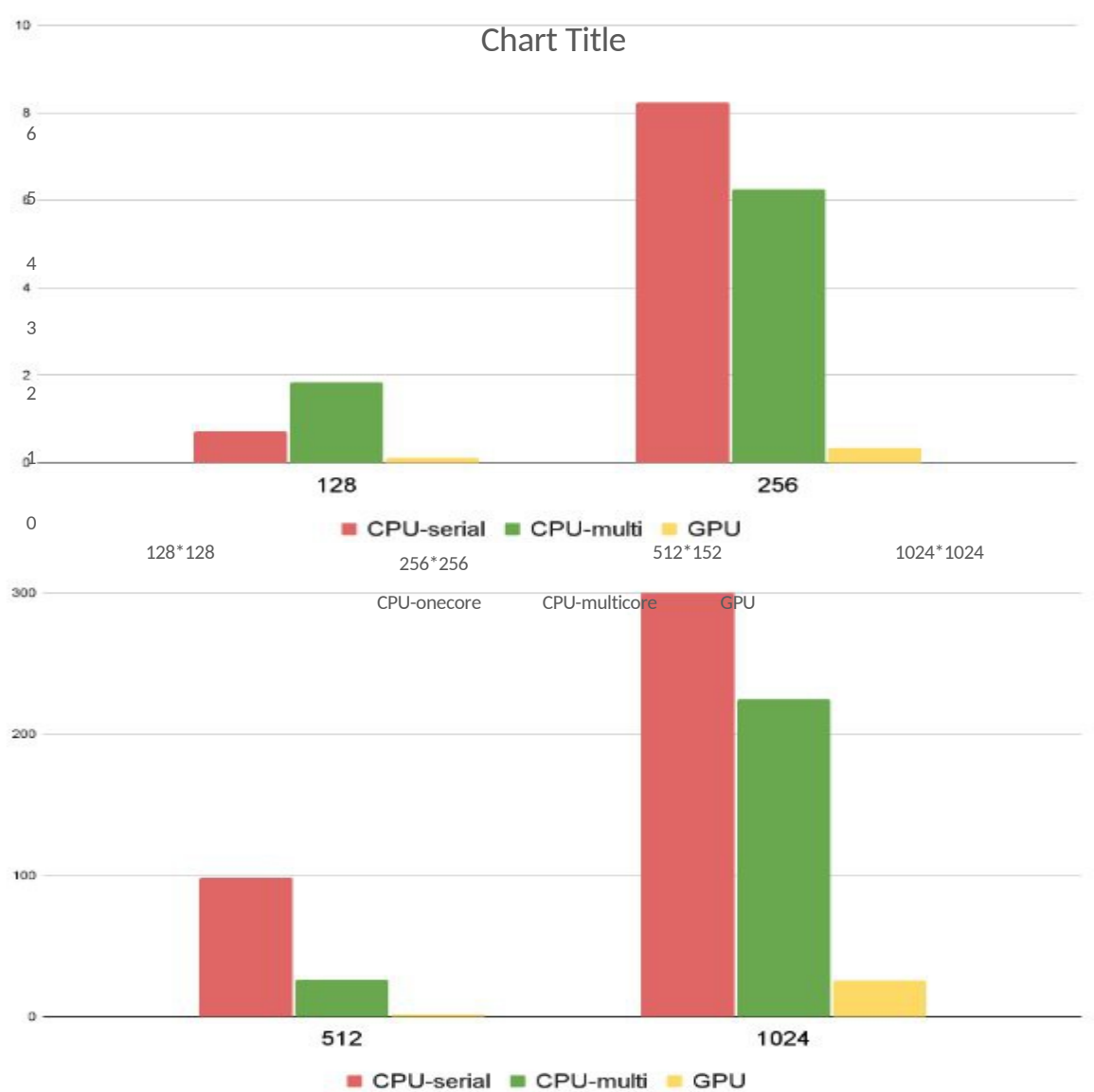
Время: 0.0284797 сек

Ошибка: 0

Итерации: 10000

Матрица:

```
10.0000 11.1111 12.2222 13.3333 14.4444 15.5556 16.6667 17.7778 18.8889 20.0000
11.1111 12.2222 13.3333 14.4444 15.5556 16.6667 17.7778 18.8889 20.0000 21.1111
12.2222 13.3333 14.4444 15.5556 16.6667 17.7778 18.8889 20.0000 21.1111 22.2222
13.3333 14.4444 15.5556 16.6667 17.7778 18.8889 20.0000 21.1111 22.2222 23.3333
14.4444 15.5556 16.6667 17.7778 18.8889 20.0000 21.1111 22.2222 23.3333 24.4444
15.5556 16.6667 17.7778 18.8889 20.0000 21.1111 22.2222 23.3333 24.4444 25.5556
16.6667 17.7778 18.8889 20.0000 21.1111 22.2222 23.3333 24.4444 25.5556 26.6667
17.7778 18.8889 20.0000 21.1111 22.2222 23.3333 24.4444 25.5556 26.6667 27.7778
18.8889 20.0000 21.1111 22.2222 23.3333 24.4444 25.5556 26.6667 27.7778 28.8889
20.0000 21.1111 22.2222 23.3333 24.4444 25.5556 26.6667 27.7778 28.8889 30.0000
```

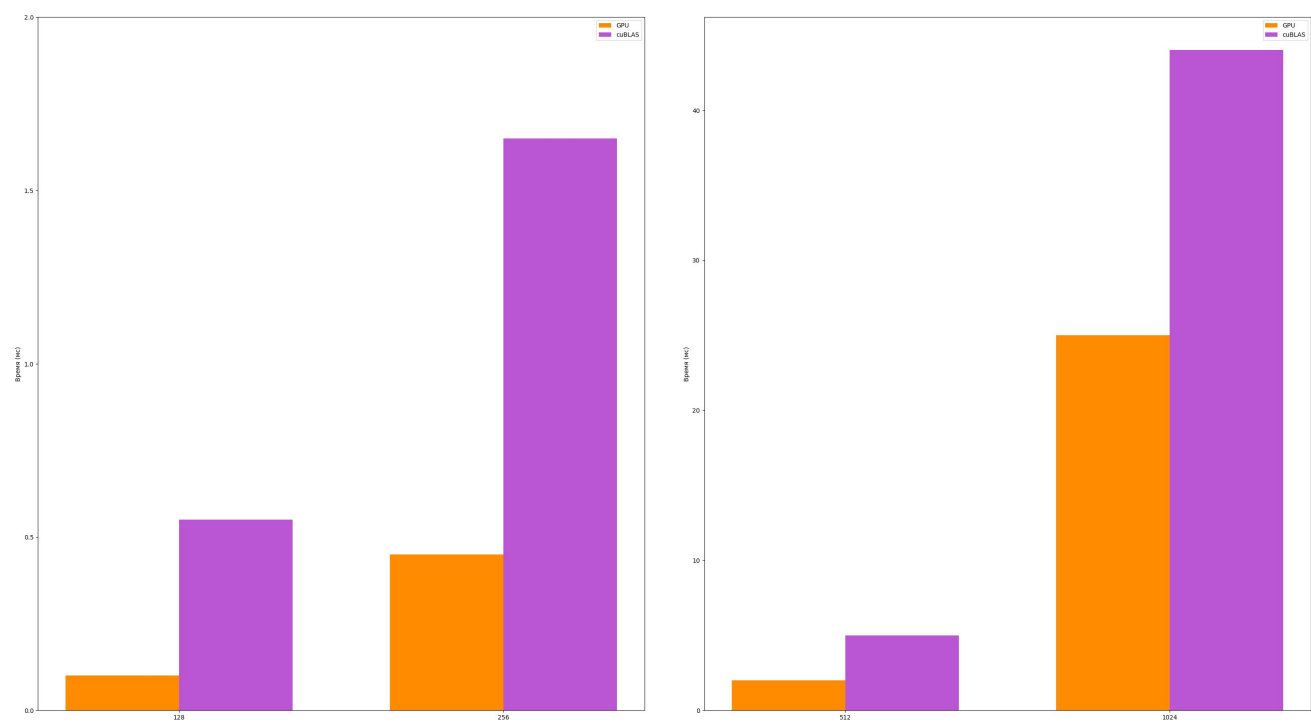


Замена редукции на функции библиотеки cuBLAS

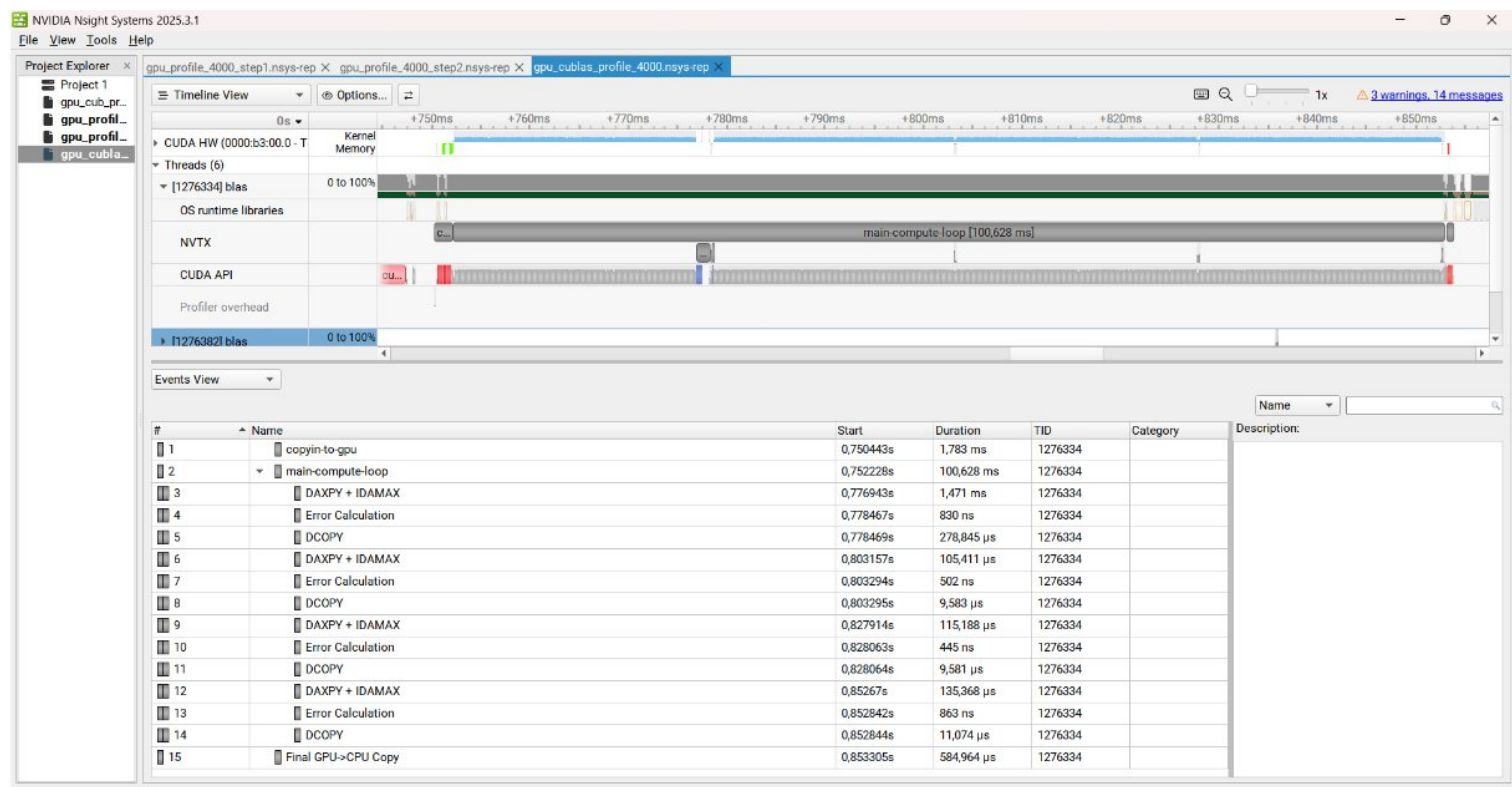
cuBLAS реализация

Размер сетки	Время выполнения	Точность	Количество итераций
128*128	0.611 сек	4.7965e-08	40000
256*256	1.677 сек	5.82721e-07	110000
512*512	5.394 сек	9.92435e-07	340000
1024*1024	33.620 сек	1.36929e-06	1000000

Сравнение программ с cuBLAS и без



Профилирование



Вывод:

Как и следовало ожидать, многопоточное выполнение на CPU показывает более высокую производительность по сравнению с одноядерным вариантом, а использование GPU позволяет добиться ещё более существенного ускорения. Тем не менее, часть преимуществ GPU нивелируется затратами на обмен данными между устройством и хостом, которые желательно минимизировать. Также стоит отметить, что операция редукции, используемая для вычисления ошибки (`max:error`), требует значительных ресурсов и оказывает заметное влияние на общее время выполнения программы.

Использование функций библиотеки cuBLAS способно значительно повысить производительность при обработке больших объемов данных, особенно если полностью заменить соответствующие фрагменты алгоритма на оптимизированные версии с применением cuBLAS. Однако в текущей реализации вычисление максимальной ошибки замедляется из-за необходимости копирования данных внутри памяти GPU, что уменьшает общий прирост скорости от использования cuBLAS.